

1) Throughput and Tail Latency (Core Metric)

Plot A — FPS vs. Scene (Bar Chart)

- Y-axis: Frames per second (FPS, higher is better)
- X-axis: Test scenes (e.g., Lego/Chair/Hotdog, Mip-NeRF 360, Tanks & Temples)
- Series: Baseline 3DGS, FlashGS, Speedy-Splat, and the proposed reduction-tree method

In addition to average FPS, minimum FPS (MinFPS) is reported to capture worst-case performance. If space permits, each method is shown with two bars: AvgFPS and MinFPS.

Rationale: Average FPS can obscure severe stalls under high depth complexity. MinFPS exposes worst-case behavior.

2) End-to-End Frame Time Breakdown

Plot B — Stacked Bar of Per-Frame Execution Time

- Y-axis: Frame time in milliseconds
- Stacks:
 1. Binning / tile assignment
 2. Sorting
 3. Gaussian evaluation
 4. Alpha blending / compositing
 5. System overhead
- X-axis: Baseline vs. proposed method

Rationale: This figure isolates the impact of the reduction-tree formulation on the compositing stage and directly reveals whether alpha blending ceases to be the dominant bottleneck.

3) Scaling with Depth Complexity

Plot C — Compositing Time vs. Number of Contributing Gaussians per Pixel

- X-axis: Average (or 90th-percentile) number of Gaussians per pixel
- Y-axis: Compositing time per frame (ms) or per tile (μ s)
- Curves: Sequential blending vs. reduction-tree blending

If possible, the corresponding speedup is also reported on a secondary axis.

Rationale: This experiment highlights the fundamental algorithmic difference between sequential and parallel blending and demonstrates where the reduction-tree formulation provides the greatest benefit (high-overdraw regions).

4) Scaling with Threads and Tile Granularity

Plot D — Performance vs. Thread Count or Tile Size

- X-axis: Block size (e.g., 64/128/256 threads) or tile size (8×8 , 16×16 , 32×32)
- Y-axis: FPS or compositing throughput (tiles per second)
- Curves: Proposed method under different configurations

Rationale: This analysis evaluates whether the approach maps efficiently to the Thor GPU architecture and whether performance continues to scale with available parallelism.

5) Energy Efficiency

Plot E — Energy per Frame and Performance per Watt

Rather than reporting power alone, efficiency is evaluated using at least one of the following metrics:

- Energy per frame:
 $\text{J/frame} = W / \text{FPS}$
- Performance per Watt:
 FPS / W

Raw power (Watts) may also be included, but always alongside J/frame.

Rationale: Power consumption alone can increase even when a method is more efficient. Energy per frame provides a fairer comparison across methods.

6) Quality–Performance Trade-off

Plot F — PSNR vs. FPS (Pareto Analysis)

- X-axis: FPS
- Y-axis: PSNR (or LPIPS where applicable)
- Points: Baseline configurations and reduction-tree configurations (with and without redundancy pruning)

Rationale: This plot illustrates whether performance improvements are obtained at the expense of image quality and whether the proposed approach lies on or near the quality–speed Pareto frontier.

7) GPU Utilization and Resource Efficiency

Plot G — Compute Utilization and Memory Bandwidth

- Metrics:
 - Compute utilization (e.g., SM occupancy or active cycles)
 - Achieved memory bandwidth (GB/s or percentage of peak)
- X-axis: Baseline vs. proposed method

Example metrics (depending on tooling):

- NVIDIA: `sm__throughput.avg.pct_of_peak_sustained_elapsed`, achieved occupancy, DRAM bandwidth (GB/s)
- General: Kernel occupancy, achieved bandwidth, L2 cache hit rate

Rationale: These measurements indicate whether the proposed method increases effective parallelism and reduces pipeline stalls.

8) Memory Footprint

Plot H — Peak GPU Memory Usage

- Y-axis: Peak GPU memory (GB)
- X-axis: Scenes or image resolutions
- Series: Baseline vs. proposed method (including variants with normals and fused Adam)

Rationale: Reduction-tree compositing may introduce temporary buffers. This experiment verifies that memory overhead remains modest and does not limit scalability.